🤖 Modelos de AA

📉 Regresión lineal

¿Qué es?

La **regresión lineal** es un modelo que busca la **relación lineal** entre:

- **atributos** (entradas)
- **etiqueta** (lo que queremos predecir)

Se usa cuando el resultado es un **número**.

Ejemplo típico:

- Peso del auto → consumo de combustible

Idea clave

Cuando graficamos los datos, muchas veces siguen una **tendencia**.
La regresión lineal intenta encontrar la **línea que mejor se ajusta** a esos puntos.

➡ Esa línea es el **modelo**.

Ecuación del modelo lineal

Forma usada en AA:

$$\hat{y} = b + w \cdot x$$

Donde:

- **ŷ** → valor predicho (salida)
- **b** → sesgo (intersección con eje Y)
- **w** → peso (pendiente de la recta)
- **x** → atributo (entrada)

➡ **w y b son parámetros del modelo** y se aprenden durante el entrenamiento.

¿Qué aprende el modelo?

Durante el entrenamiento, el modelo:

- **NO cambia los datos**
- **NO cambia los atributos**
- **AJUSTA los pesos (w) y el sesgo (b)**

➡ El objetivo es que la línea represente lo mejor posible los datos.

Predicción

Una vez entrenado:

- Se le da un valor de entrada
- Usa la ecuación
- Devuelve una predicción numérica

Ejemplo:

- Peso del auto → consumo estimado

Modelos con múltiples atributos

La regresión lineal puede usar **más de un atributo**:

$$\hat{y} = b + w_1x_1 + w_2x_2 + w_3x_3 + ...$$

Cada atributo:

- Tiene su propio **peso**
- Aporta información distinta

Ejemplo de atributos:

- Cilindrada
- Potencia
- **...**

📉 Regresión lineal — Pérdida

La **pérdida (loss)** es una medida numérica que indica **qué tan incorrectas son las predicciones del modelo**.
Mide la **distancia entre el valor predicho y el valor real**.

Objetivo del entrenamiento:
ajustar los parámetros del modelo para **minimizar la pérdida**.

Distancia de pérdida

La pérdida se enfoca en la **magnitud del error**, no en su signo.
No importa si el modelo predice de más o de menos, solo importa **qué tan lejos está del valor real**.

Por eso, al calcular la pérdida:

- se elimina el signo,
- se trabaja siempre con distancias positivas.

Formas de eliminar el signo

Las dos más usadas son:

- **Valor absoluto:** $|y - \hat{y}|$
- **Diferencia al cuadrado:** $(y - \hat{y})^2$

Tipos de pérdida en regresión lineal

- **L1:**
  Suma de los valores absolutos de los errores.
- **MAE (Error Absoluto Medio):**
  Promedio de la pérdida L1.
  Es fácil de interpretar porque está en la misma escala que los datos.
- **L2:**
  Suma de los errores elevados al cuadrado.
- **ECM / MSE (Error Cuadrático Medio):**
  Promedio de la pérdida L2.
  Penaliza fuertemente los errores grandes.

- **RECM / RMSE:**
  Raíz cuadrada del ECM. Más interpretable para humanos que el ECM.

Diferencia entre L1 y L2

- Para errores grandes, **L2 los amplifica mucho más**.
- Para errores pequeños, **L2 los hace aún más pequeños**.

➡ Esto hace que L2 sea muy sensible a errores grandes.

Valores atípicos (outliers)

Un valor atípico puede ser:

- un dato fuera de lo común,
- o una predicción muy alejada del valor real.

Efecto según la pérdida:

- **ECM / L2:** el modelo se acerca más a los outliers.
- **MAE / L1:** el modelo es más robusto y los ignora mejor.

Elección de la función de pérdida

- **ECM:** útil si se quiere penalizar fuertemente errores grandes.
- **MAE:** útil si se busca estabilidad y menor influencia de valores atípicos.

➡ En la práctica, la pérdida se calcula como el **promedio sobre todos los ejemplos**.

📉 Regresión lineal — Descenso de gradientes

El **descenso de gradientes** es un método matemático que permite encontrar, de forma **iterativa**, los valores del **peso (w)** y el **sesgo (b)** que producen la **menor pérdida posible**.

➡ Es el algoritmo que **ajusta automáticamente** los parámetros del modelo durante el entrenamiento.

Idea general

El modelo:

- empieza con valores iniciales de **w** y **b** (generalmente cercanos a cero),
- evalúa qué tan mala es la predicción,
- y corrige los parámetros **paso a paso** para mejorar.

Este proceso se repite muchas veces, en lo que se llaman **iteraciones**.

Pasos del descenso de gradientes

En cada iteración, el algoritmo:

1. Calcula la **pérdida** con los valores actuales de peso y sesgo.
2. Determina **en qué dirección** moverlos para reducir la pérdida.
3. Ajusta ligeramente **w** y **b** en esa dirección.
4. Repite el proceso hasta que la pérdida ya no pueda disminuir más.

Curva de pérdida

Durante el entrenamiento se suele observar la **curva de pérdida**, que muestra:

- eje X: iteraciones
- eje Y: pérdida

Comportamiento típico:

- al principio la pérdida baja rápido,

- luego baja cada vez más lento,
- finalmente se estabiliza.

➡ Cuando la curva se estabiliza, decimos que el modelo **convergió**.

Convergencia del modelo

Un modelo **converge** cuando:

- ya encontró valores de **w** y **b** que producen la menor pérdida,
- y las iteraciones siguientes solo generan cambios mínimos.

Aunque el modelo casi nunca encuentra el mínimo exacto, se acerca **lo suficiente** como para considerarlo óptimo.

Funciones convexas

En regresión lineal, la función de pérdida es **convexa**.

Esto significa que:

- existe **un único mínimo global**,
- el descenso de gradientes **no se queda atrapado en mínimos locales**.

➡ Por eso, cuando converge, podemos estar seguros de que el modelo encontrado es el mejor posible para esos datos.

Intuición visual

El proceso se puede imaginar como:

- una pelota rodando cuesta abajo,
- hasta llegar al punto más bajo,
- donde ya no hay pendiente para seguir bajando.

Ese punto representa los valores óptimos de **peso y sesgo**.

📉 Regresión lineal — Hiperparámetros

Los **hiperparámetros** son valores que **vos definís antes del entrenamiento** y que controlan **cómo aprende el modelo**.

➡ No hay que confundirlos con los **parámetros**:

- **Parámetros:** pesos (w) y sesgo (b) → los calcula el modelo.
- **Hiperparámetros:** reglas del entrenamiento → los elegís vos.

Tasa de aprendizaje (learning rate)

La **tasa de aprendizaje** determina **qué tan grande es cada paso** que da el modelo al ajustar los pesos y el sesgo durante el descenso de gradientes.

- **Muy baja:**
  el modelo aprende muy lento y necesita muchas iteraciones.
- **Muy alta:**
  el modelo se pasa del mínimo y **no converge** (la pérdida sube y baja).
- **Adecuada:**
  permite converger rápido y de forma estable.

Idea clave:
El ajuste de los parámetros es proporcional al **gradiente × tasa de aprendizaje**.

Ejemplo:

- gradiente = 2.5
- tasa de aprendizaje = 0.01
  → cambio del parámetro = 0.025

➡ No existe una tasa ideal universal: **depende del problema y de los datos**.

Tamaño del lote (batch size)

El **tamaño del lote** indica **cuántos ejemplos se usan antes de actualizar los pesos y el sesgo**.

No siempre se usa todo el conjunto de datos porque puede ser muy grande.

🔹 **Descenso de gradientes estocástico (SGD)**

- tamaño de lote = 1
- actualiza los parámetros **después de cada ejemplo**
- es rápido pero **muy ruidoso**

➡ La pérdida fluctúa bastante durante el entrenamiento.

🔹 **Descenso de gradientes por minilotes**

- usa pequeños subconjuntos de datos (ni 1 ni todo)
- promedia los gradientes del lote
- es un equilibrio entre estabilidad y velocidad

➡ Es el método más usado en la práctica.

Épocas (epochs)

Una **época** significa que el modelo:

- vio **todos los ejemplos del conjunto de entrenamiento una vez**.

Ejemplo:

- 1.000 datos
- tamaño de lote = 100
  → 10 iteraciones = 1 época

➡ Normalmente se necesitan **varias épocas** para que el modelo converja.

- más épocas → mejor ajuste (hasta cierto punto)
- pero también → más tiempo de entrenamiento

Relación entre lote y actualizaciones

- **Lote completo:**
  actualiza una vez por época.

- **SGD:**
  actualiza una vez por ejemplo.

- **Minilote:**
  actualiza una vez por lote.

➡ Esto afecta cuántas veces se modifican los pesos y el sesgo.

Ruido y generalización

Aunque el ruido parezca algo negativo:

- una **pequeña cantidad de ruido** puede ayudar al modelo
- a **generalizar mejor**
- y evitar soluciones malas.

Esto es especialmente importante en modelos más complejos como **redes neuronales**.

🔐 Regresión logística

La **regresión logística** es un tipo de modelo de aprendizaje automático que se utiliza cuando el objetivo **no es predecir un valor numérico continuo**, sino **estimar la probabilidad de que ocurra un determinado resultado**.

A diferencia de la regresión lineal, que se usa para problemas como la eficiencia de combustible de un automóvil, la regresión logística permite responder preguntas del tipo:

- ¿Lloverá hoy?
- ¿Este correo electrónico es spam?

En estos casos, el modelo no devuelve un número cualquiera, sino una **probabilidad**, que indica qué tan probable es que el resultado ocurra.

Este tipo de modelo es especialmente útil para problemas donde la salida representa una **decisión** o **clasificación**, generalmente asociada a dos posibles resultados.

🔐 Regresión logística — Cálculo de probabilidades

La regresión logística es un modelo de aprendizaje automático que se usa cuando el objetivo es **estimar una probabilidad**.

➡ La salida del modelo **siempre está entre 0 y 1**.

Se usa en problemas como:
- detección de spam
- predicción de lluvia
- aprobación de pagos

Uso de la probabilidad

La probabilidad que devuelve el modelo se puede usar de dos formas:

- Usarla directamente

Ejemplo:
salida = 0.932
→ 93.2% de probabilidad de que sea spam

- Convertirla en una decisión binaria

Spam / No spam
Sí / No

➡ En estos apuntes se trabaja **con la probabilidad tal cual**.

Función sigmoidea

La regresión logística usa una función especial para garantizar que la salida sea una probabilidad.

➡ Esa función es la **función sigmoidea** (o logística).

Características:
- tiene forma de S
- nunca vale 0 ni 1
- siempre está entre 0 y 1

Comportamiento de la sigmoide

Cuando la entrada:
- es muy grande → la salida se acerca a 1
- es muy negativa → la salida se acerca a 0
- vale 0 → la salida es 0.5

➡ Esto permite interpretar la salida como probabilidad.

Salida lineal (log-odds)

Antes de aplicar la sigmoide, el modelo calcula una combinación lineal:

z = combinación de atributos (multiplica cada atributo por su peso) + sesgo

Donde:
z → salida lineal (log-odds)
w → pesos aprendidos
x → atributos
b → sesgo

➡ El valor de z **no está limitado**: puede ser muy grande o muy chico.

Transformación con la sigmoide

Para obtener la predicción final:
➡ el valor z se pasa por la función sigmoidea.

Resultado:
➡ una probabilidad entre 0 y 1

➡ La sigmoide “aplasta” valores extremos:
z muy negativo → probabilidad cercana a 0
z muy positivo → probabilidad cercana a 1

Intuición visual

La ecuación lineal genera una recta.
La función sigmoidea curva esa recta en forma de S.

Ejemplo:
z = −10 → probabilidad ≈ 0
z = 0 → probabilidad = 0.5
z = 5 → probabilidad ≈ 1

🔐 Regresión logística — Pérdida y regularización

Los modelos de regresión logística se entrenan de forma muy similar a los de regresión lineal, pero con **dos diferencias clave**:

🔹 Usan **pérdida logística** en lugar de pérdida al cuadrado.
🔹 La **regularización** es fundamental para evitar el sobreajuste.

Pérdida logística

En regresión lineal se usa la **pérdida al cuadrado (L2)**.
Esto funciona bien cuando la relación entre entrada y salida es **lineal**.

➡ En un modelo lineal un cambio constante en la entrada, produce un cambio constante en la salida

---

Problema con la pérdida cuadrática en regresión logística

La regresión logística **no es lineal**.

➡ Usa una **función sigmoidea**, que tiene forma de S.

Características de la sigmoide:

- cerca de z = 0 → pequeños cambios en z producen **grandes cambios** en la salida
- para z muy grandes o muy chicos → la salida cambia **muy poco**

➡ La tasa de cambio **no es constante**.

Por qué no usar pérdida cuadrática

Cuando la salida se acerca a:

- 0
- 1

la pérdida cuadrática:
🔹 requiere muchísima precisión numérica
🔹 es ineficiente
🔹 no refleja bien el error real

➡ Esto la hace poco adecuada para la sigmoide.

Pérdida logística (log loss)

Por eso, la regresión logística usa la **pérdida logística**.

➡ Idea clave:

- No mide solo la distancia, mide el **logaritmo del error**

➡ Penaliza fuertemente:

- Predicciones muy seguras, que son incorrectas

Qué usa la pérdida logística

Para cada ejemplo:

- la etiqueta real es **0 o 1**
- la predicción es un valor entre **0 y 1**

La pérdida se calcula:

- Comparando la etiqueta real, con la probabilidad predicha

Interpretación intuitiva

- predecir 0.99 cuando la etiqueta es 0 → **error enorme**
- predecir 0.51 cuando la etiqueta es 0 → **error menor**

➡ La pérdida logística castiga mucho la “confianza equivocada”.

Regularización en regresión logística

La regularización es un mecanismo que:
🔹 penaliza modelos demasiado complejos
🔹 evita el **sobreajuste**

➡ En regresión logística es **especialmente importante**.

Qué pasa sin regularización

Sin regularización:

- el modelo puede seguir ajustándose
- la pérdida sigue bajando
- los pesos crecen demasiado

➡ El modelo se adapta **demasiado a los datos de entrenamiento**.

Regularización L2

La regularización L2:

- penaliza pesos grandes
- mantiene el modelo más simple
- mejora la generalización

➡ Es la más usada en regresión logística.

Interrupción anticipada (early stopping)

Otra forma de regularizar es:
detener el entrenamiento **antes de tiempo**.

Se corta cuando:

- la pérdida deja de mejorar de forma significativa
- el modelo empieza a sobreajustar

🧮 Clasificación

La clasificación es la tarea de predecir a cuál de un conjunto de clases (categorías) pertenece un ejemplo.

A diferencia de la regresión, donde el resultado es un valor numérico, en clasificación el resultado es una **categoría**.

De probabilidad a clase

En regresión logística, el modelo utiliza la función sigmoidea para transformar la salida del modelo en un valor entre 0 y 1.
Ese valor representa una **probabilidad**.

Ejemplo:
- Un correo electrónico puede tener un 75% de probabilidad de ser spam.

Cuando el objetivo no es una probabilidad sino una categoría, esa probabilidad se usa para asignar una clase.

Clasificación binaria

La clasificación binaria consiste en predecir una de dos clases posibles.

Ejemplo típico:
- Spam
- No spam

En este caso, un modelo de regresión logística que predice probabilidades se utiliza para realizar una decisión de clasificación.

🧮Clasificación — Umbrales y matriz de confusión

Para convertir la probabilidad predicha por un modelo de regresión logística en una categoría, se define un **umbral de clasificación**.

Los ejemplos con una probabilidad **mayor** al umbral se asignan a la **clase positiva**.
Los ejemplos con una probabilidad **menor** al umbral se asignan a la **clase negativa**.

Ejemplo:
- Umbral = **0.5** → probabilidades 0.99 y 0.51 se clasifican como spam
- Umbral = **0.95** → solo 0.99 se clasifica como spam

El umbral **0.5 no siempre es adecuado**, especialmente cuando:
- las clases están desbalanceadas
- el costo de un tipo de error es mayor que el otro

Matriz de confusión

En un clasificador binario existen **cuatro resultados posibles**, que se organizan en la **matriz de confusión**, comparando la predicción del modelo con la verdad fundamental.

- **Verdadero positivo (VP)**
  Spam clasificado correctamente como spam.

- **Falso positivo (FP)**
  Correo legítimo clasificado erróneamente como spam.

- **Falso negativo (FN)**
  Spam clasificado erróneamente como no spam.

- **Verdadero negativo (VN)**
  Correo legítimo clasificado correctamente como no spam.

El total de cada fila representa las **predicciones del modelo**, mientras que el total de cada columna representa los **valores reales**, independientemente de la predicción.

Datos desbalanceados

Un conjunto de datos está **desbalanceado** cuando la cantidad de ejemplos positivos reales es muy diferente de la cantidad de negativos reales.

En estos casos, la elección del umbral tiene un impacto significativo en el desempeño del modelo.

Efecto del umbral

Cambiar el umbral modifica la cantidad de:

- Verdaderos positivos
- Falsos positivos
- Verdaderos negativos
- Falsos negativos

Diferentes umbrales generan distintos compromisos entre detectar correctamente la clase positiva y evitar errores.

🧮Clasificación — Exactitud, recuperación, precisión y métricas relacionadas

Los **verdaderos y falsos positivos y negativos** se utilizan para calcular distintas **métricas de evaluación** de un modelo de clasificación.
La métrica más adecuada depende de:
- la tarea específica
- el costo de los errores
- si el conjunto de datos está equilibrado

Todas las métricas de esta sección se calculan con un **umbral fijo** y cambian cuando el umbral cambia.

Exactitud

La **exactitud** es la proporción de todas las clasificaciones que fueron correctas, tanto positivas como negativas.

Mide qué fracción del total de ejemplos fue clasificada correctamente.

Un modelo perfecto no tiene falsos positivos ni falsos negativos y alcanza una exactitud de **1.0 (100%)**.

En conjuntos de datos **equilibrados**, la exactitud puede servir como una medida general de calidad del modelo.
Sin embargo, en conjuntos de datos **desequilibrados**, puede ser engañosa.

Ejemplo:
Si solo el 1% de los datos pertenece a la clase positiva, un modelo que siempre predice negativo puede tener **99% de exactitud** y aun así ser inútil.

Recuperación (tasa de verdaderos positivos)

La **recuperación** mide la proporción de positivos reales que el modelo clasifica correctamente como positivos.

También se conoce como:
- tasa de verdaderos positivos
- probabilidad de detección

En el ejemplo de spam, indica qué fracción de los correos spam fue detectada correctamente.

Un modelo perfecto no tiene falsos negativos y alcanza una recuperación de **1.0**.

En conjuntos de datos desequilibrados donde los positivos reales son pocos, la recuperación es más informativa que la exactitud, especialmente cuando los **falsos negativos son costosos**.

Tasa de falsos positivos

La **tasa de falsos positivos (FPR)** es la proporción de negativos reales que el modelo clasifica incorrectamente como positivos.

También se interpreta como la **probabilidad de falsa alarma**.

Un modelo perfecto no tiene falsos positivos y su FPR es **0.0**.

Cuando hay muy pocos negativos reales, esta métrica pierde utilidad.

Precisión

La **precisión** mide la proporción de predicciones positivas que realmente son positivas.

En el ejemplo de spam, indica qué fracción de los correos clasificados como spam realmente lo eran.

Un modelo perfecto no tiene falsos positivos y alcanza una precisión de **1.0**.

En conjuntos de datos muy desequilibrados con pocos positivos reales, la precisión también puede ser poco informativa.

Relación entre precisión y recuperación

La **precisión** mejora cuando disminuyen los falsos positivos.
La **recuperación** mejora cuando disminuyen los falsos negativos.

Cambiar el umbral genera un compromiso entre ambas métricas:
- aumentar el umbral → menos falsos positivos, más falsos negativos
- disminuir el umbral → más falsos positivos, menos falsos negativos

Por eso, mejorar una suele empeorar la otra.

Elección de la métrica

La métrica a optimizar depende del problema:

- **Exactitud**
  Útil solo como indicador general en datos equilibrados.
  No recomendable como única métrica en datos desequilibrados.

- **Recuperación**
  Útil cuando los falsos negativos son más costosos.

- **Tasa de falsos positivos**
  Útil cuando los falsos positivos son más costosos.

- **Precisión**
  Útil cuando es crítico que las predicciones positivas sean correctas.

🧮Clasificación — ROC y AUC

Las métricas vistas anteriormente se calculan para **un solo umbral**.
Para evaluar el rendimiento de un modelo en **todos los umbrales posibles**, se utilizan la **curva ROC** y la **AUC**.

Curva ROC

La **curva ROC** (característica operativa del receptor) es una representación visual del rendimiento del modelo para distintos umbrales.

Para construirla:
- se calcula la **tasa de verdaderos positivos (TPR)**
- se calcula la **tasa de falsos positivos (FPR)**
- se grafica **TPR (eje Y)** contra **FPR (eje X)**

Cada punto de la curva corresponde a un umbral distinto.

Un **modelo perfecto** alcanza:
- TPR = 1.0
- FPR = 0.0

Ese comportamiento se representa cerca del punto **(0, 1)**.

Área bajo la curva (AUC)

La **AUC** mide el área bajo la curva ROC.

Representa la **probabilidad** de que el modelo asigne una puntuación mayor a un ejemplo positivo elegido al azar que a un ejemplo negativo elegido al azar.

- **AUC = 1.0**
  Modelo perfecto: siempre ordena correctamente positivos por encima de negativos.

- **AUC = 0.5**
  Modelo equivalente al azar, como lanzar una moneda.

En el ejemplo de spam:
Una AUC de 1.0 significa que un correo spam aleatorio recibe siempre una probabilidad mayor que un correo legítimo aleatorio, independientemente del umbral.

Interpretación del ROC

Un clasificador aleatorio produce una curva ROC que es una **línea diagonal** de (0,0) a (1,1).

Cuanto más se acerca la curva ROC al punto **(0,1)**, mejor es el modelo.

Una curva con mayor **AUC** indica mejor capacidad de discriminación entre clases.

Uso de ROC y AUC

La **AUC** es útil para:
- comparar distintos modelos
- evaluar rendimiento cuando el conjunto de datos está aproximadamente equilibrado

El modelo con mayor AUC suele ser el mejor.

Los puntos de la curva ROC más cercanos a (0,1) representan **umbrales con mejor desempeño**, pero el umbral óptimo depende del costo relativo de los errores.

Elección del umbral usando la ROC

Según el problema:

- Si los **falsos positivos** son muy costosos, conviene un umbral con **FPR baja**, aunque disminuya la TPR.
- Si los **falsos negativos** son muy costosos, conviene un umbral que **maximice la TPR**, aunque aumente la FPR.
- Si los costos son similares, se busca un equilibrio entre TPR y FPR.

🧮Clasificación — Sesgo de predicción

El **sesgo de predicción** es una verificación rápida que permite detectar problemas en el modelo o en los datos de entrenamiento en etapas tempranas.

Se define como la **diferencia entre**:
- la **media de las predicciones** del modelo
- la **media de las etiquetas de verdad fundamental**

Ejemplo:
Si en el conjunto de datos el **5%** de los correos electrónicos son spam, la media de las etiquetas es **0.05**.
Un modelo bien calibrado debería predecir, en promedio, que **el 5%** de los correos son spam.

Si ambas medias coinciden, el modelo tiene **sesgo de predicción cero**.
Esto no garantiza que el modelo sea correcto, pero indica que no hay un problema evidente de desbalance en las predicciones.

Sesgo de predicción distinto de cero

Si el modelo predice, por ejemplo, que **el 50%** de los correos son spam cuando en realidad solo lo es el **5%**, existe un problema claro.

Cualquier **diferencia significativa** entre la media de las predicciones y la media de las etiquetas sugiere la presencia de sesgo de predicción.

Posibles causas del sesgo de predicción

El sesgo de predicción puede deberse a:

- **Sesgos o ruido en los datos**, incluido un muestreo sesgado del conjunto de entrenamiento
- **Regularización excesiva**, que simplifica demasiado el modelo
- **Errores en la canalización de entrenamiento**
- **Atributos insuficientes** para resolver la tarea

🧮Clasificación — clasificación de clases múltiples

La **clasificación de clases múltiples** es una extensión de la clasificación binaria a problemas con **más de dos clases**.

Si cada ejemplo solo puede pertenecer a **una clase**, el problema puede abordarse como varios problemas de **clasificación binaria**, agrupando una clase contra todas las demás.

Conversión a problemas binarios

En un problema con clases **A, B y C**:

- Se puede crear un primer clasificador binario que distinga **(A + B)** frente a **C**.
- Luego, un segundo clasificador binario puede separar **A** de **B** dentro del grupo (A + B).

Este procedimiento permite resolver un problema multiclase usando clasificadores binarios.

Ejemplo de clasificación multiclase

Un ejemplo típico es el **reconocimiento de dígitos escritos a mano**, donde el modelo debe decidir qué número, del **0 al 9**, representa una imagen.

Clasificación de etiquetas múltiples

Si un ejemplo puede pertenecer a **más de una clase al mismo tiempo**, el problema se denomina **clasificación de etiquetas múltiples**, y no clasificación multiclase tradicional.